

지 지형 — 개발 세계는 어떻게 나뉘나

골 골목 — 폴더는 왜 이렇게 갈라졌나

문 문패 — 파일들은 무슨 신분인가

만든다 이해한다

구조를 그린다 — 개발 세계의 지도 · 오늘은 아무것도 만들지 않습니다. 대신 전부 이해합니다

10차엔 소스를 감췄습니다 — 오늘은 다 펼칩니다

여러분은 관찰로 뽑힌 반입니다. 이제 겉이 아니라 속을 봅니다.

10차 — 소스를 감췄다

- 실행 파일 하나만 드렸다 (소스 비공개)
- 겉과 결과물만으로 속을 추론했다
- 측정한 것 = 사용자로서의 관찰력

오늘 — 속 지도를 펼친다

- 교재 = 여러분이 매주 점수 받던 그 채점기
- 폴더 하나하나, 파일 하나하나 — 왜 있는지
- 얻는 것 = 개발 세계를 읽는 지도

진단받던 그 도구가, **오늘의 교재**입니다.

개발 지식 시리즈 — 내 앱 설계 3종 세트

격주로 초급반과 교차합니다. 지금 잡힌 계획은 3회 — 이후는 진행을 보고 이어집니다.

12

구조를 그린다

웹 작동 원리 · 폴더 구조 · 좋은 구조 → 숙제: 내 앱 구조도

14

화면을 그린다

HTML·CSS·JS · 컴포넌트 · UX/UI → 숙제: 내 앱 화면도

16

데이터를 그린다

DB · 테이블 · 보안 → 숙제: 내 앱 데이터도

세 장을 모으면 「내 앱 설계 3종 세트」 — 그다음에, 진짜로 만듭니다.

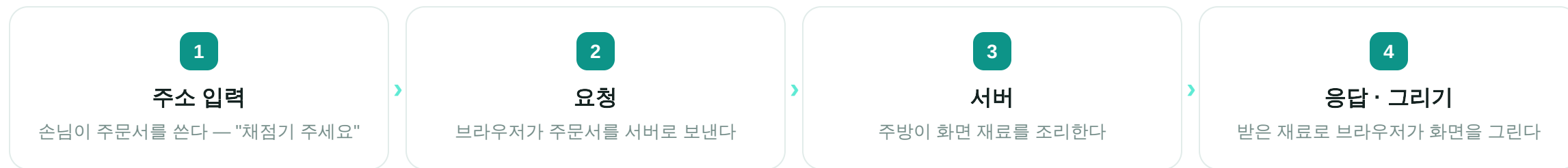
매 회차는 **직전 숙제 리뷰**로 시작합니다 — 오늘의 숙제가 14차의 첫 재료.

"모든 파일에는 **자리**가 있고,
그 자리에는 **이유**가 있다"

여러분은 이미 전부 해봤습니다 — 만들고, 배포하고, 키까지 숨겨봤죠.
이미 다 해봤습니다. 오늘은 이름을 배웁니다.

주소창의 여정 — 엔터 후 3초 동안 생긴 일

채점기 주소를 치고 엔터를 눌렀을 때, 화면이 뜨기까지.



손님(브라우저)과 주방(서버) — 이 경계선이 오늘 지도의 첫 번째 선입니다.

인터넷의 모든 것이 이 **요청** → **응답** 한 왕복의 반복입니다.

개발 세계의 큰 지형 — 4겹의 실명

데이터 · 동작 · 화면 · 연결 — 그 네 칸을 개발 세계는 이렇게 부릅니다.

화면 →

프론트엔드

손님 앞의 **홀** — 보이는 전부. 버튼·글씨·색깔 (UX/UI)

동작 →

백엔드

손님이 못 들어가는 **주방 안쪽** — 계산과 판단

데이터 →

DB

꺾다 켜도 남는 **창고** — 데이터베이스

연결 →

API

프로그램끼리의 **주문 창구** — 정해진 양식으로 묻고 답한다

이미 다 해봤습니다. **오늘은 이름을 배웁니다.**

채점기 하나에 다 있다

9차의 5요소 기억나시죠 — 몸·기억·무대·창고·맥박. 오늘은 그 '몸' 속으로 들어갑니다.

Next.js 몸

출과 주방을 한 몸에 — 화면(프론트엔드)과 동작(백엔드)이 한 폴더에 사는 조립 키트

Supabase 창고

DB — 리더보드 점수가 사는 곳. **우리 폴더 밖**, 다른 회사 서버에 있다 (그래서 '연결'이 필요)

Railway 무대

배포 — 조리된 앱이 24시간 서 있는 곳. push하면 맥박처럼 자동으로 올라간다

오늘 지도의 대상은 이 '**몸**' 폴더 — 여러분이 매주 쓰던 채점기의 속입니다.

루트 — 폴더를 열면 처음 보이는 것들

프롬프트 채점/

- └ **package.json** ← 재료 목록 — 이름·부품·명령어
- └ 설정 파일들 ← 주방 세팅 (tsconfig 등 4개)
- └ **.env.local** ← 열쇠 서랍 🗝️ (git엔 절대 안 올라감)
- └ .gitignore ← 문지기 — 못 올라갈 것들의 명단
- └ **README.md** · **CLAUDE.md** ← 문패 2장 — 사람용 · AI용
- └ docs/ · public/ ← 문서 · 그대로 내보내는 것들
- └ **node_modules/** ← 창고 — 내가 안 만듦 (제일 무거움)
- └ **src/** ← 내가 만든 것의 전부

내가 만든 건 **절반뿐** — 나머지는 도구와 관례가 만들었습니다. 그 정체를 지금 밝힙니다.

내가 안 만든 파일들의 정체 — 요리로 읽기

목록 `package.json` **재료 목록** — 이 앱의 이름, 필요한 부품(패키지), 실행 명령이 다 적혀 있다

참고 `node_modules` **장 봐온 참고** — `npm install` 한 번에 목록대로 채워진다

조리 `빌드` 재료를 **조리해 내놓을 상태**로 — `npm run build` → `.next` 폴더

열쇠 `.env` **열쇠 서랍** — API 키 같은 비밀값. 코드와 분리, GitHub에 절대 안 올린다

문지기 `.gitignore` 참고·열쇠·조리 결과물을 **GitHub에 못 올라가게** 막는 명단

Q. 그럼 — **node_modules**를 지우면 앱은 죽을까요? (손 들어봅시다)

문패 2장 — 사람용과 AI용

6차에서 만들었던 그 두 파일 — 문서도 구조의 일부입니다.

README.md — 사람용 문패

- 뭐 하는 앱인지, 어떻게 켜는지
- 폴더를 연 사람이 제일 먼저 읽는 파일
- 대문자 이름 = "나부터 읽어라"는 관례

CLAUDE.md — AI용 문패

- 규칙 · 합정 · 히스토리를 AI에게
- AI가 작업 전에 읽고 사고를 예방한다
- 6차에서 우리가 직접 만든 그 파일

채점기의 CLAUDE.md는 **39KB — A4 30장**. 11개 강의의 역사가 전부 적혀 있습니다.

src — 내가 만든 것의 다섯 골목

src/

└ app/	← 주소가 있는 것들의 동네	화면	연결
└ components/	← 화면 부품 26개 - 레고	화면	
└ lib/	← 동작 - 채점 기준·세션 처리	동작	
└ types/	← 계약 - 주고받는 것의 약속	계약	
└ data/	← content.ts 한 파일 = 225KB	데이터	

한 폴더가 두 겹에 걸치기도 합니다 — **지도는 단순화**입니다. 실물은 경계가 흐려요.
그래도 지도 없이 다니는 것보다 백 배 낫습니다.

폴더가 곧 주소다

app 골목의 규칙 — 폴더 경로가 그대로 URL이 됩니다.

<code>/</code> 랜딩	<code>app/page.tsx</code> — 강의 소개 첫 화면
<code>/practice</code> 실습	<code>app/practice/</code> — 채점 연습하던 그 페이지
<code>/play/ABCD</code> 세션	<code>app/play/[code]/</code> — 대괄호 = 변하는 주소 (세션 코드마다 다른 방)
<code>/api/score</code> 창구	<code>app/api/score/</code> — 화면 없는 골목 . 주소는 있는데 보이는 건 없다 — 채점 백엔드

Q. 그럼 채점기의 **화면(주소)**은 모두 몇 개일까요? — 매주 쓰신 앱입니다.

같은 원리, 다른 생김새

파이썬 프로젝트를 열어도 길을 잃지 않는 법 — 이름만 바꿉니다.

JavaScript — 우리 채점기

- `package.json` — 재료 목록
- `node_modules/` — 참고
- `src/` — 소스 (내가 만든 것)

Python — 이름만 다르다

- `requirements.txt` — 재료 목록
- `venv/` — 참고
- `main.py` · `src/` — 소스

이름은 달라도 **자리는 같다** — 재료 목록 · 참고 · 소스. 구조의 원리는 언어를 넘습니다.

좋은 구조의 원칙 5

원칙 ①

한 폴더 = 한 책임

이유를 한 줄로 못 쓰는 폴더는 없어야 할 폴더다

원칙 ②

화면과 동작을 섞지 않는다

채점 로직은 화면(components)이 아니라 lib·api에 산다

원칙 ③

주고받는 것은 계약으로

types/ — 점수의 생김새를 약속. 2차 I/O 계약의 코드판

원칙 ④ ★

데이터와 코드를 섞지 않는다

강의 내용은 전부 content.ts 한 파일 — 그래서 코드 수술 없이 강의가 추가된다

원칙 ⑤

문패를 단다

사람용(README)과 AI용(CLAUDE.md) — 문서도 구조다

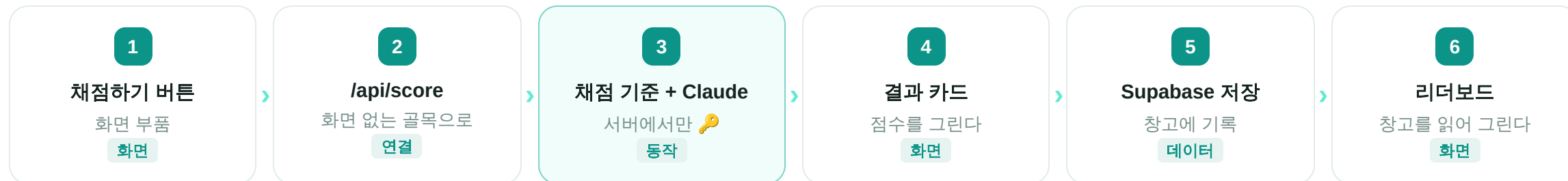
속제에서

이 다섯이 채점 기준

여러분의 구조도는 이 원칙으로 스스로 검사합니다

증거 — 이 채점기에 강의가 **11개** 추가되는 동안, 대부분의 회차에서 고친 파일은 **content.ts 하나**였습니다.

채점 버튼 하나의 여정 — 수백 번 누른 그 버튼



열쇠(.env)는 **3번 — 서버 쪽에서만** 열립니다. 브라우저에 두면 도난당해요. (왜인지는 16차에서)

이 여정이 그려지면 — **오늘 지도는 완성**입니다.

숙제 — 내 앱 폴더 구조 설계

만들고 싶은 앱 하나를 정해, 오늘 배운 원칙대로 폴더 트리를 설계지에 그려웁니다. 종이와 펜이면 충분합니다.

합격 기준 ①

최상위 폴더 3~6개

더 많으면 책임이 안 갈라진 것

합격 기준 ②

모든 폴더에 "왜" 한 줄

이유를 못 쓰는 폴더는 지운다

합격 기준 ③

4겹 태그

폴더마다 데·동·화·연 중 하나 이상

보너스 — "내가 안 만들 파일" 자리 1개 이상 비워두기 (node_modules처럼 도구가 만들어줄 자리)

14차는 이 종이로 시작합니다 — 반드시 지참! 다음 시간엔 이 구조 위에 화면을 그립니다.

모든 파일에는 자리가 있고, 그 자리에는 **이유**가 있다

오늘 처음으로 — 지도를 들고 개발 세계에 들어왔습니다 · 설계지 꼭 챙겨 오세요
수고하셨습니다!